

Diseño de Arquitectura de Solución – Sistema de Banca por Internet BP

Ejercicio práctico – Arquitectura de Soluciones

Andrés Eduardo Cárdenas Jaramillo – Arquitecto de Soluciones

Índice de contenido

Resumen ejecutivo.....	2
Alcance y supuestos.....	2
Consideraciones normativas y de cumplimiento.....	3
Estilo arquitectónico: microservicios + BFF.....	4
Proveedor cloud: AWS.....	4
Backend de microservicios: Node.js + NestJS.....	4
Backend de microservicios: Java + Spring Boot.....	5
SPA Web: React.....	5
Aplicación móvil multiplataforma: React Native (evaluado frente a Flutter).....	5
Patrón API Gateway + capa de integración.....	5
Comunicación híbrida: síncrona (REST) + asíncrona (eventos).....	6
Patrón de resiliencia: Circuit Breaker + Retry + Timeout.....	6
Arquitectura de las aplicaciones front-end y móvil.....	6
Diagramas C4.....	7
Diagrama de Contexto (Nivel 1).....	7
Diagrama de Contenedores (Nivel 2).....	8
Diagrama de Componentes (Nivel 3).....	9
Arquitectura de autenticación y autorización.....	10
Onboarding con verificación biométrica.....	10

Diagrama de secuencia: onboarding + autenticación.....	12
Diseño de la solución de auditoría.....	12
Arquitectura de acceso a datos.....	13
Infraestructura cloud AWS, alta disponibilidad y DR.....	14
Alta disponibilidad y tolerancia a fallos.....	14
Recuperación ante desastres (DR).....	15
Seguridad.....	15
Monitoreo, excelencia operativa y auto-healing.....	16
Gestión de costos.....	16
Arquitectura desacoplada, reusable y extensible.....	16
Conclusiones.....	17

Resumen ejecutivo

Este documento presenta la arquitectura de solución propuesta, modelada bajo el **C4 Model** (Contexto, Contenedores y Componentes), junto con las decisiones técnicas tomadas, su justificación, los patrones de arquitectura aplicados, y las consideraciones de seguridad, cumplimiento normativo, alta disponibilidad, recuperación ante desastres, monitoreo y costos.

La arquitectura propuesta se apoya en un estilo de **microservicios desacoplados**, orquestados a través de un patrón **API Gateway + Backend for Frontend (BFF)**, con comunicación híbrida síncrona (REST/HTTPS) y asíncrona (eventos), desplegada sobre **AWS** con un enfoque de **alta disponibilidad multi-AZ** y **recuperación ante desastres multi-región (warm standby)**.

Alcance y supuestos

El diseño de arquitectura – hasta nivel de Componentes del modelo C4 – para:

- Consulta de histórico de movimientos, saldos y productos del cliente.
- Transferencias y pagos entre cuentas propias e interbancarias.

- Onboarding digital de nuevos clientes con verificación de identidad biométrica.
- Autenticación de usuarios mediante un proveedor de identidad (IdP) corporativo ya existente, configurado con OAuth 2.0.
- Notificación de movimientos por al menos dos canales.
- Auditoría de todas las acciones del cliente sobre el sistema.

Supuestos explícitos (dado que el enunciado no los especifica, se enuncian para que sean validados por el negocio):

- El país de operación no se indica en el ejercicio; se documentan como referencia los marcos regulatorios típicos de banca digital en Latinoamérica, con foco en Colombia (Superintendencia Financiera de Colombia – SFC) por ser el marco más representativo para este tipo de ejercicio, complementado con estándares internacionales.
- BP ya cuenta con un Core Bancario y un sistema complementario de información de clientes, ambos con API expuestas (o exponibles) para integración.
- El producto de identidad/OAuth 2.0 ya existe (mencionado en el enunciado); este documento se limita a recomendar el flujo de autenticación y su integración, no su implementación interna.
- Existe presupuesto para servicios administrados en AWS, priorizando *time-to-market* y menor carga operativa sobre control total de infraestructura.
- Se analiza con un equipo grande de desarrolladores y roles técnicos con conocimiento de negocio bien definidos y equipos segmentados por dominio.

Consideraciones normativas y de cumplimiento

Por tratarse de una entidad financiera que procesa datos personales, biométricos y transaccionales, la arquitectura debe considerar.

Protección de datos personales. Ley de Habeas Data / protección de datos personales (p. ej. Ley 1581 de 2012 y Decreto 1377 de 2013 en Colombia; equivalentes GDPR/LGPD en otras jurisdicciones). Los datos biométricos se tratan como **datos sensibles**: requieren consentimiento explícito, cifrado en tránsito y en reposo, y minimización (no se persiste la imagen cruda más allá de lo estrictamente necesario para el proceso de verificación).

Seguridad y ciberseguridad para entidades vigiladas. Circulares del regulador financiero sobre requerimientos mínimos de seguridad y calidad para canales digitales, y sobre gestión de riesgo de ciberseguridad (en Colombia, Circulares Externas de la SFC como la 007/008 y la 042 de 2021 sobre riesgo operacional y continuidad del negocio, que exige RTO/RPO definidos y planes de continuidad probados).

Prevención de lavado de activos (AML/PLAFT). Los flujos de onboarding y transferencias deben soportar controles de debida diligencia (KYC/CDD), listas restrictivas y monitoreo transaccional (SARLAFT o equivalente).

Estándares de seguridad de pagos. PCI-DSS si en algún punto se manejan datos de tarjetas.

Gestión de seguridad de la información. ISO/IEC 27001 como marco de referencia para el SGSI, e ISO/IEC 27701 para privacidad.

Estándares internacionales de referencia técnica. PSD2/Open Banking (buenas prácticas de APIs seguras para banca), NIST SP 800-63 (niveles de garantía de identidad digital, aplicable al diseño de onboarding e IAL/AAL), y OWASP ASVS/MASVS como checklist de seguridad para las aplicaciones web y móvil.

Estilo arquitectónico: microservicios + BFF

Decisión: Diseñar una arquitectura basada en microservicios. Estos estarán organizados por dominio (Clientes, Movimientos, Transferencias, Onboarding, Identidad, Notificaciones, Auditoría), El conjunto de los microservicios estarán expuestos a través de un patrón *Backend for Frontend*.

- *Justificación 1:* Los canales web y móvil tienen necesidades de payload, latencia y ciclos de release distintos; un BFF por canal permite evolucionar cada experiencia sin acoplar el modelo de datos expuesto ni forzar despliegues conjuntos.
- *Justificación 2:* El enunciado exige alta disponibilidad, tolerancia a fallos y desacoplamiento; los microservicios permiten escalar y desplegar de forma independiente el servicio que necesite cambios (crítico, con SLA estrictos) sin afectar a la operación de los demás.
- *Alternativa evaluada:* monolito modular. Se descarta ya que dificulta el escalamiento independiente y aumenta el radio de impacto de un fallo o un despliegue, dos requisitos explícitos del ejercicio (HA y tolerancia a fallos). Además cada equipo es encargado de ser él dueño de su negocio y toma decisiones de cambios y despliegue y de operaciones de mantenimiento.

Proveedor cloud: AWS

Decisión: infraestructura sobre AWS.

- *Justificación 1:* AWS ofrece servicios maduros y muy probados en la industria para todos los requerimientos en un mismo ecosistema: identidad (Cognito/IAM), bases de datos relacionales con replicación global (Aurora Global Database), mensajería (EventBridge/MSK), y observabilidad (CloudWatch/X-Ray), reduciendo la integración entre proveedores.
- *Justificación 2:* Route 53 + Aurora Global Database + S3 Cross-Region Replication dan una ruta directa y bien documentada para el patrón *warm standby* multi-región que exige el ejercicio (HA + DR), con RPO/RTO medibles y probados en la industria bancaria.
- *Nota:* la arquitectura lógica (C4) es agnóstica de nube; los mismos contenedores podrían desplegarse en Azure (API Management, AKS, Azure SQL/Cosmos DB, Azure AD B2C) si BP ya tuviera ese estándar corporativo.

Backend de microservicios: Node.js + NestJS

- *Justificación 1:* NestJS aporta una arquitectura modular (módulos, controladores, providers con inyección de dependencias) que facilita mantener bajo acoplamiento y

alta cohesión entre los microservicios y sus adaptadores, alineado con el principio de segmentación de responsabilidades exigido en la rúbrica.

- *Justificación 2:* el modelo de I/O no bloqueante de Node.js es adecuado para servicios que son, en su mayoría, orquestadores de llamadas a APIs externas (Core Bancario, sistema complementario, red interbancaria), donde el cuello de botella es de red y no de cómputo.

Backend de microservicios: Java + Spring Boot

- *Justificación 1:* Este lenguaje se usará para operaciones más complejas que requieran operaciones, transformación de información o servicios que requieren un cuidado especial antes de la comunicación con el Core bancario.
- *Justificación 2:* Usar java y Spring-boot permiten desarrollos rápidos sin demasiada gestión de configuración, además contiene servidores web integrados lo que facilita las pruebas, el despliegue y la observabilidad. .

SPA Web: React

- *Justificación 1:* Ecosistema maduro para *token handling* seguro vía patrón BFF (librerías de manejo de sesión basadas en cookies httpOnly), reduciendo superficie de ataque XSS frente a mantener tokens en localStorage.

Aplicación móvil multiplataforma: React Native (evaluado frente a Flutter)

Opción A – React Native: comparte lenguaje (TypeScript) y parte de la lógica de negocio/validaciones con la SPA; acceso a APIs nativas (Keychain/Keystore, biometría de plataforma) vía módulos nativos maduros; comunidad amplia de librerías certificadas para KYC/onboarding (SDKs de proveedores de verificación biométrica suelen publicar primero para React Native e iOS/Android nativo).

Opción B – Flutter: mejor performance percibida en animaciones complejas y un único motor de renderizado (Skia) que garantiza consistencia visual entre plataformas; Dart es un lenguaje distinto al del resto del stack.

- *Justificación de la elección (React Native):* Al compartir TypeScript con los servicios de interacción el backend (NestJS) y la SPA (React), se reduce el costo de mantener ecosistemas distintos y facilita reutilizar validadores, tipos (DTOs) y clientes HTTP entre SPA y app móvil.
- *Justificación 2:* La mayoría de proveedores de reconocimiento facial/liveness (requeridos para el onboarding) publican SDKs nativos con *wrappers* oficiales o de comunidad para React Native antes que para Flutter, reduciendo riesgo de integración en el punto más sensible del flujo.

Patrón API Gateway + capa de integración

- *Justificación 1:* Centraliza *cross-cutting concerns* (autenticación, rate limiting, validación de esquema, WAF) en un único punto, evitando que cada microservicio implemente estas responsabilidades.

- *Justificación 2:* permite versionar y exponer un contrato estable a los canales cliente mientras los microservicios internos evolucionan, cumpliendo el requisito de arquitectura desacoplada y reusable para futuros componentes.

Comunicación híbrida: síncrona (REST) + asíncrona (eventos)

- *Justificación 1:* Las operaciones que requieren respuesta inmediata al cliente (consultar saldo, ejecutar una transferencia) usan REST/HTTPS síncrono; las que son de propagación (notificar, auditar) usan eventos, evitando que la latencia de un canal de notificación externo (SMS/email) bloquee la respuesta transaccional al usuario.
- *Justificación 2:* El bus de eventos desacopla productores de consumidores: se puede añadir un nuevo consumidor (p. ej. un futuro motor de scoring antifraude) sin modificar el Servicio de Transferencias, cumpliendo el requisito de arquitectura reusable/extensible.

Patrón de resiliencia: Circuit Breaker + Retry + Timeout

- *Justificación 1:* El Core Bancario y el sistema complementario son dependencias externas fuera del control del equipo de Banca Digital; un *circuit breaker* evita que una degradación de esos sistemas se propague en cascada (efecto dominó) hacia la app del cliente.
- *Justificación 2:* Junto con *fallback* (p. ej. servir saldo desde cache con marca de “última actualización”), mejora la disponibilidad percibida sin comprometer la integridad de las transferencias (que no aceptan fallback, solo *fail-fast*).

Arquitectura de las aplicaciones front-end y móvil

- **SPA Web (React):** consume exclusivamente al BFF Web, nunca a los microservicios directamente. El manejo del *Access Token* se delega al BFF (patrón *Token Handler* / BFF de seguridad), que lo guarda en una cookie httpOnly, Secure, SameSite=Strict; el navegador nunca tiene acceso al token en JavaScript, mitigando robo de tokens vía XSS.
- **App Móvil (React Native):** consume al BFF Mobile. El *Access Token* y *Refresh Token* se guardan en almacenamiento seguro del dispositivo (Keychain en iOS, Keystore/EncryptedSharedPreferences en Android), nunca en almacenamiento plano.
- Ambos *frontends* comparten una librería interna de tipos/DTOs y validaciones (publicada como paquete privado npm) para evitar duplicación de lógica y reducir inconsistencias entre canales.
- Todas las llamadas usan HTTPS/TLS 1.2+; certificado *pinning* recomendado en la app móvil para mitigar ataques *man-in-the-middle*.

Diagramas C4

Diagrama de Contexto (Nivel 1)

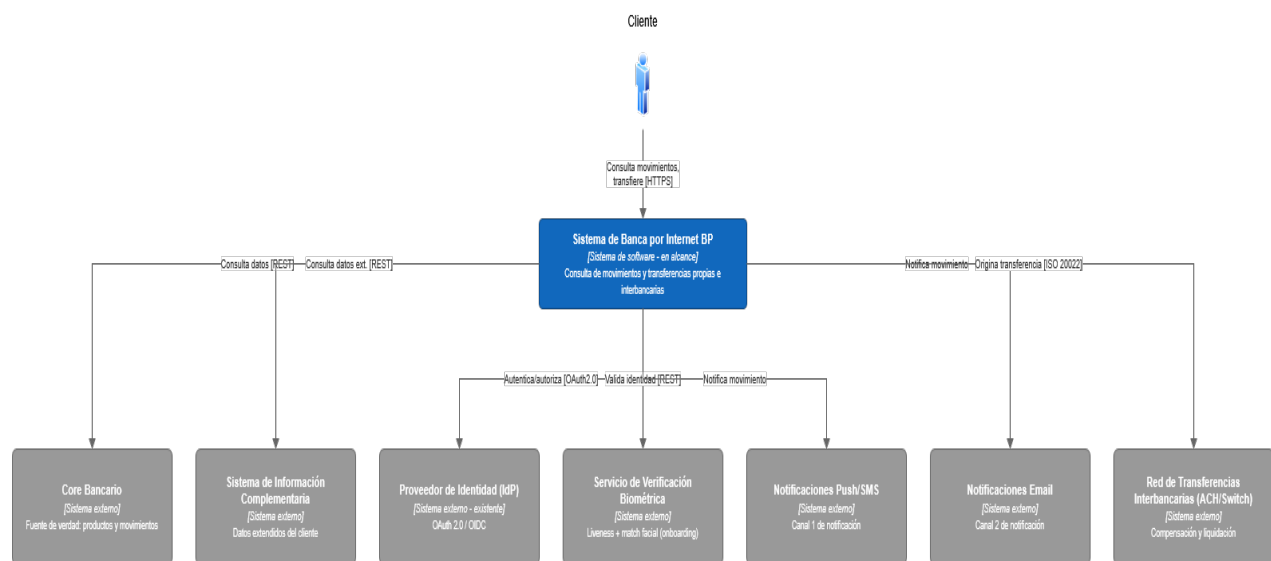


Diagrama de Contexto

El **Cliente** interactúa únicamente con el “Sistema de Banca por Internet BP”. Este, a su vez, se integra con seis sistemas externos: el **Core Bancario** (fuente de verdad de productos y movimientos), el **Sistema de Información Complementaria** (datos extendidos del cliente), el **Proveedor de Identidad** (autenticación OAuth 2.0, ya existente), el **Servicio de Verificación Biométrica** (onboarding), dos **proveedores de notificaciones** (push/SMS y email, cumpliendo el requisito normativo de notificar movimientos por al menos dos canales) y la **Red de Transferencias Interbancarias**. (Para mayor detalle o ver mejor la imagen dirigirse a la imagen que está en el mismo repositorio o al draw.io)

Diagrama de Contenedores (Nivel 2)

Expone las aplicaciones, servicios, bases de datos y mensajería, incluyendo los componentes cloud de AWS empleados.

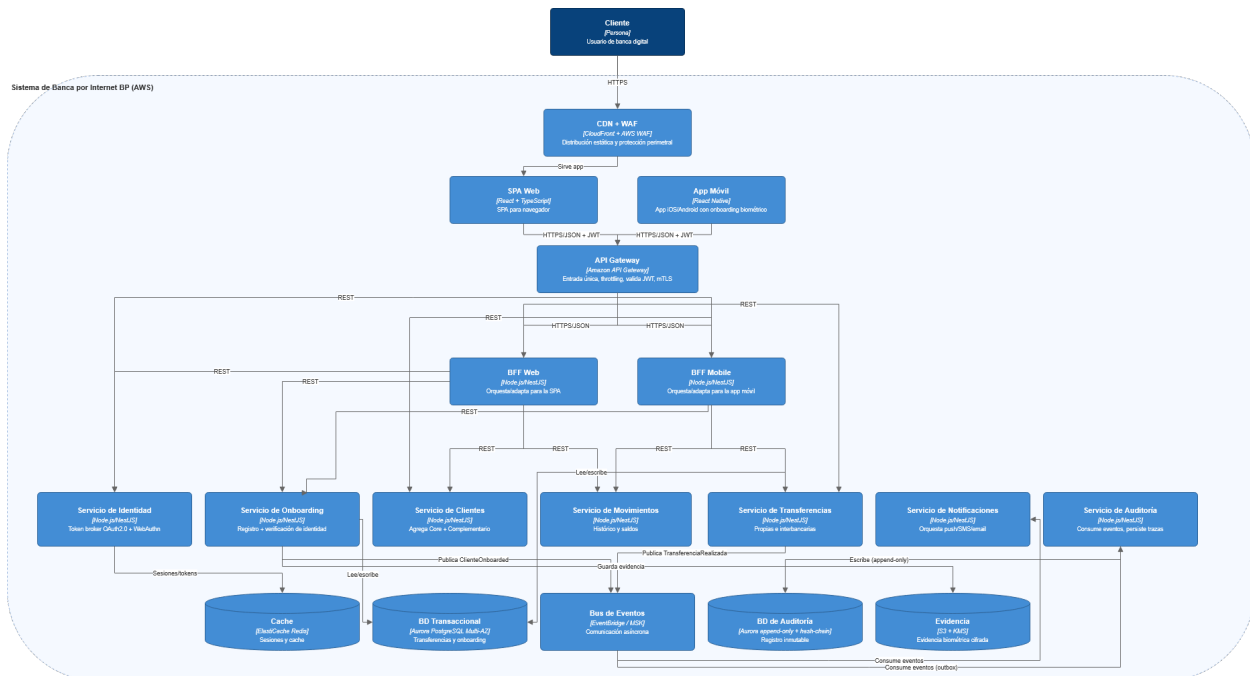


Diagrama de Contenedores

Puntos clave del diseño de contenedores:

- **CDN + WAF** (CloudFront + AWS WAF) sirve la SPA y filtra tráfico malicioso antes de llegar al API Gateway.
- **API Gateway** es el único punto de entrada para ambos canales; valida el JWT emitido por el IdP antes de enrutar al BFF correspondiente.
- **BFF Web** y **BFF Mobile** son contenedores independientes: permiten adaptar payloads y política de tokens por canal sin acoplar sus ciclos de despliegue.
- Siete microservicios de dominio (Identidad, Onboarding, Clientes, Movimientos, Transferencias, Notificaciones, Auditoría), cada uno con responsabilidad única.
- Un **bus de eventos** (Amazon EventBridge / Amazon MSK) conecta Transferencias, Onboarding y Clientes con Notificaciones y Auditoría de forma asíncrona.
- Persistencia poliglota: **Aurora PostgreSQL** (transaccional), **Aurora PostgreSQL con tablas append-only** (auditoría), **ElastiCache Redis** (sesiones/cache) y **S3+KMS** (evidencia biométrica cifrada). (Para mayor detalle o ver mejor la imagen dirigirse a la imagen que está en el mismo repositorio o al draw.io)

Diagrama de Componentes (Nivel 3)

Hace zoom sobre la **capa de integración (API Gateway + BFF)**, que es explícitamente el punto que el enunciado pide detallar: aquí se resuelven las llamadas hacia los tres servicios iniciales (Clientes, Movimientos, Transferencias).

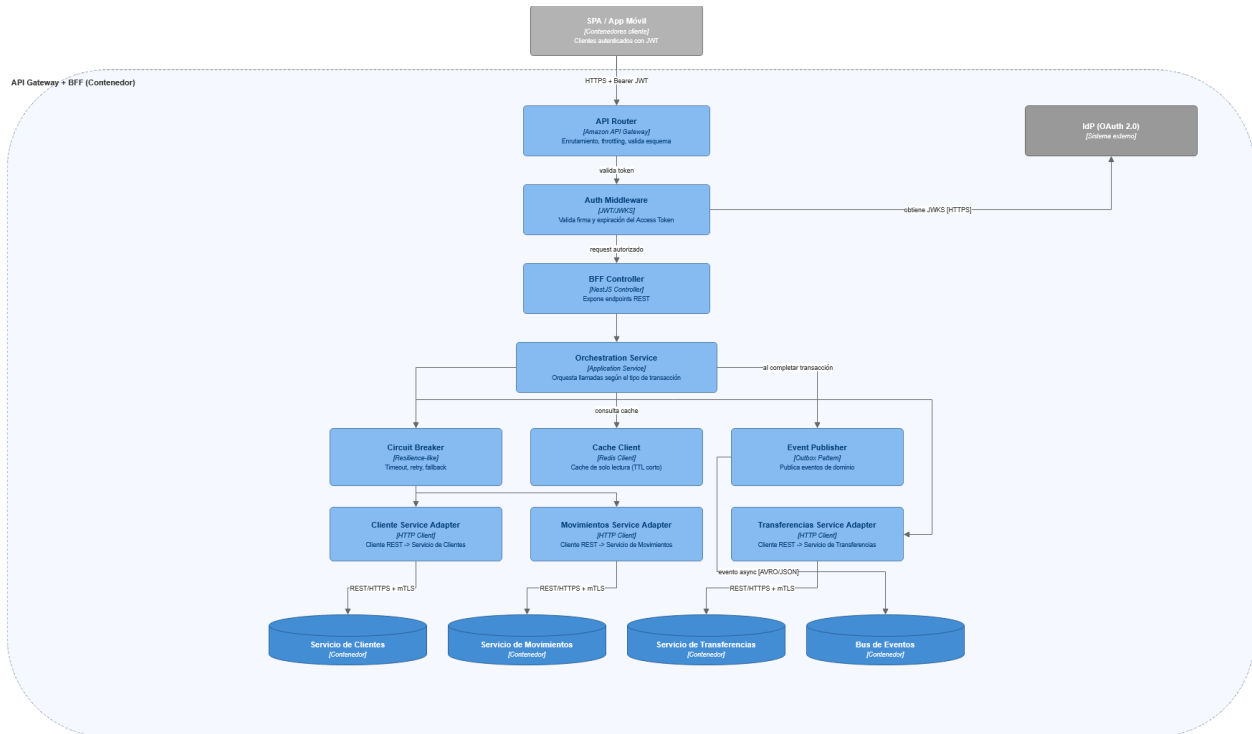


Diagrama de Componentes

Componentes destacados y su responsabilidad:

- **API Router:** enrutamiento, *throttling* y validación de esquema de entrada (OpenAPI).
- **Auth Middleware:** valida firma y expiración del *Access Token* contra el JWKS publicado por el IdP (OAuth 2.0 / OIDC), sin necesidad de llamar al IdP en cada request.
- **Orchestration Service:** orquesta, por tipo de transacción, las llamadas a uno o varios de los tres servicios (patrón *Facade*).
- **Circuit Breaker:** envuelve las llamadas salientes con timeout, reintentos con *backoff* exponencial y apertura de circuito ante fallas sostenidas.
- **Cache Client:** sirve consultas de solo lectura (saldo, movimientos) desde Redis con TTL corto, reduciendo carga sobre el Core Bancario.
- **Event Publisher (Outbox):** publica eventos de dominio de forma transaccional (ver sección de Auditoría).

Servicios adicionales propuestos más allá de los tres iniciales, para mejorar rendimiento y calidad de respuesta al cliente:

- **Servicio de Identidad** dedicado (token broker), para no acoplar la validación de tokens a cada microservicio de negocio.

- **Servicio de Onboarding** independiente de Clientes, porque su ciclo de vida (alta, verificación biométrica) y sus SLA de seguridad son distintos a los de una consulta transaccional.
- **Servicio de Auditoría** como consumidor asíncrono, para que la latencia de escritura del log de auditoría nunca bloquee una transferencia.
- Un futuro **servicio de agregación/BFF de reportes** (fuera del alcance inicial) que consume eventos históricos sin tocar los servicios transaccionales, aprovechando el desacoplamiento vía bus de eventos. (Para mayor detalle o ver mejor la imagen dirigirse a la imagen que está en el mismo repositorio o al draw.io)

Arquitectura de autenticación y autorización

El enunciado indica que la compañía ya cuenta con un producto de identidad configurable para OAuth 2.0, por lo que **no voy a diseñar el IdP**, solo recomiendo el flujo y su integración.

Authorization Code con PKCE (RFC 6749 + RFC 7636), combinado con el patrón Backend for Frontend / Token Handler para la SPA.

- Para la **app móvil**: Authorization Code + PKCE es el flujo recomendado por la propia IETF para aplicaciones nativas (RFC 8252, *OAuth 2.0 for Native Apps*), usando *custom tabs* / *ASWebAuthenticationSession* en lugar de un *WebView* embebido (para no exponer credenciales a la app), y guardando los tokens en Keychain/Keystore.
- Para la **SPA**: se descarta el flujo *Implicit* (deprecado por la propia OAuth 2.0 Security Best Current Practice, RFC 9700) y también se evita mantener el *Access Token* en JavaScript (*localStorage/sessionStorage*), por su exposición a robo vía XSS. En su lugar, el **BFF Web actúa como cliente OAuth confidencial**: es él quien ejecuta el intercambio *Authorization Code + PKCE* con el IdP, y expone al navegador únicamente una cookie de sesión *httpOnly/Secure*. Este patrón es el recomendado actualmente por el grupo de trabajo OAuth de IETF para aplicaciones basadas en navegador (*OAuth 2.0 for Browser-Based Apps*).
- *Justificación 1*: PKCE mitiga la interceptación del *authorization code* incluso en canales no completamente confiables (apps nativas, redirecciones del navegador).
- *Justificación 2*: delegar el manejo de tokens al BFF reduce la superficie de ataque a nivel de navegador, que es el vector más común de robo de sesión en banca digital (XSS/token exfiltration).
- Los *Access Tokens* son de vida corta (5-15 min); la renovación se hace con *Refresh Token* rotativo (uso único, revocado si se detecta reuso – *refresh token rotation*), reduciendo el impacto de un token filtrado.

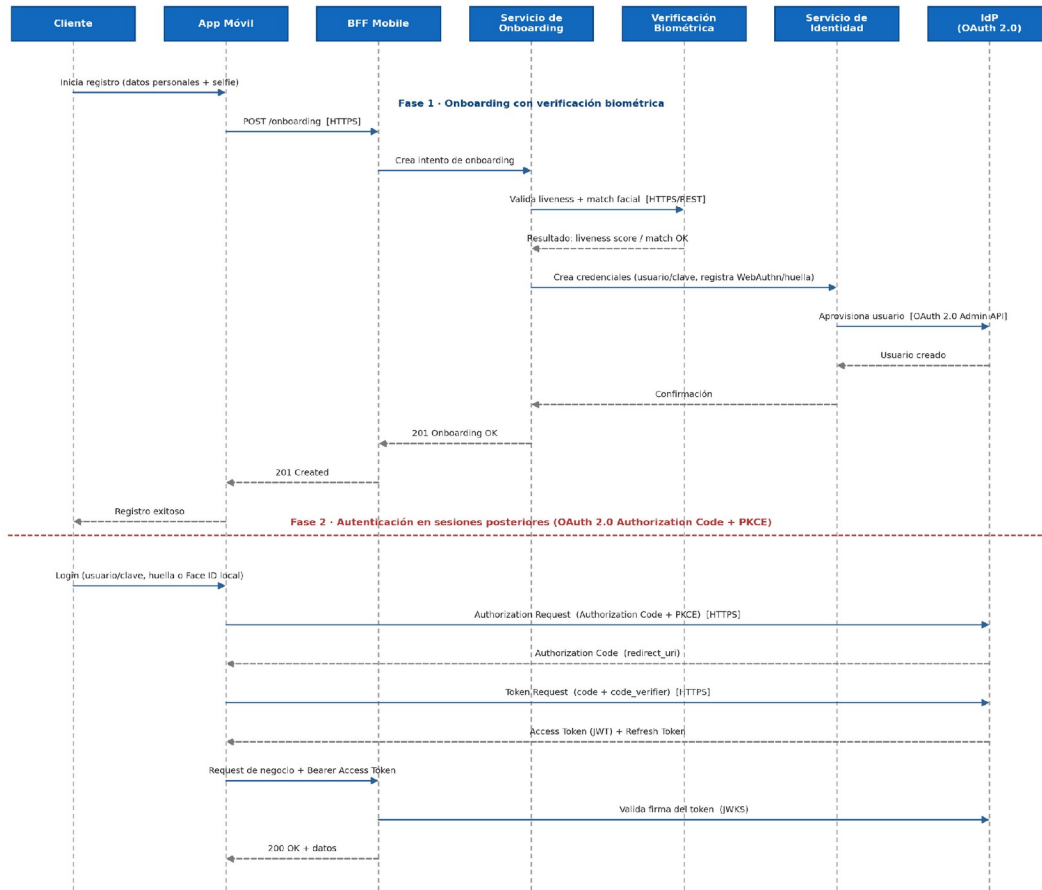
Onboarding con verificación biométrica

El flujo de autorización/autenticación debe incorporar el reconocimiento facial como parte del **Onboarding**, y habilitar métodos alternativos para el acceso posterior.

Diseño propuesto:

1. **Registro inicial:** captura de datos personales + documento de identidad + *selfie* con prueba de vida (*liveness detection*).
2. **Verificación biométrica:** el Servicio de Onboarding delega en un proveedor especializado la validación de *liveness* y el *match* facial contra el documento (build vs. buy: se recomienda *comprar* esta capacidad – p. ej. Amazon Rekognition Face Liveness o un proveedor KYC especializado como Onfido/Incode – en lugar de construirla, dado que requiere certificación anti-*spoofing* bajo estándares como ISO/IEC 30107-3, algo costoso de desarrollar y mantener in-house).
3. **Creación de credenciales:** superada la verificación, el Servicio de Identidad crea el usuario en el IdP y **registra un segundo factor biométrico local** mediante **WebAuthn/FIDO2**, usando el enclave seguro del dispositivo (Face ID/Touch ID/huella). Esta es la recomendación de herramienta de industria: WebAuthn evita transmitir o almacenar datos biométricos crudos en el backend – el dispositivo solo certifica una firma criptográfica.
4. **Acceso posterior:** el usuario puede autenticarse con usuario/clave, con el biométrico local (WebAuthn) registrado en el onboarding, o repitiendo verificación facial remota solo en operaciones de alto riesgo (p. ej. cambio de dispositivo, elevación de límites de transferencia – *step-up authentication*).
 - *Justificación 1:* separar “verificación de identidad” (una vez, en onboarding, con proveedor especializado) de “autenticación recurrente” (WebAuthn local) reduce fricción para el usuario y superficie de exposición de datos biométricos.
 - *Justificación 2:* WebAuthn/FIDO2 es phishing-resistant por diseño (el reto está atado al origen/dominio), a diferencia de OTP por SMS, elevando el nivel de garantía de autenticación (AAL2/AAL3 según NIST SP 800-63B) sin fricción adicional para el usuario.

Diagrama de secuencia: onboarding + autenticación



Secuencia de Onboarding y Autenticación

La fase 1 muestra el registro con verificación biométrica y aprovisionamiento del usuario en el IdP. La fase 2 muestra el flujo de autenticación en sesiones posteriores, con Authorization Code + PKCE y validación de token vía JWKS en cada request al BFF. (Para mayor detalle o ver mejor la imagen dirigirse a la imagen que está en el mismo repositorio o al draw.io)

Diseño de la solución de auditoría

Requisito: una base de datos de auditoría que registre todas las acciones del cliente, con un mecanismo de persistencia para clientes frecuentes.

Patrón propuesto: Transactional Outbox + Event Sourcing parcial, con un almacén de auditoría *append-only* y verificable (*tamper-evident log*).

Diseño:

1. Cada microservicio de negocio (Transferencias, Onboarding, Clientes) escribe su cambio de estado y, **en la misma transacción de base de datos**, inserta el evento

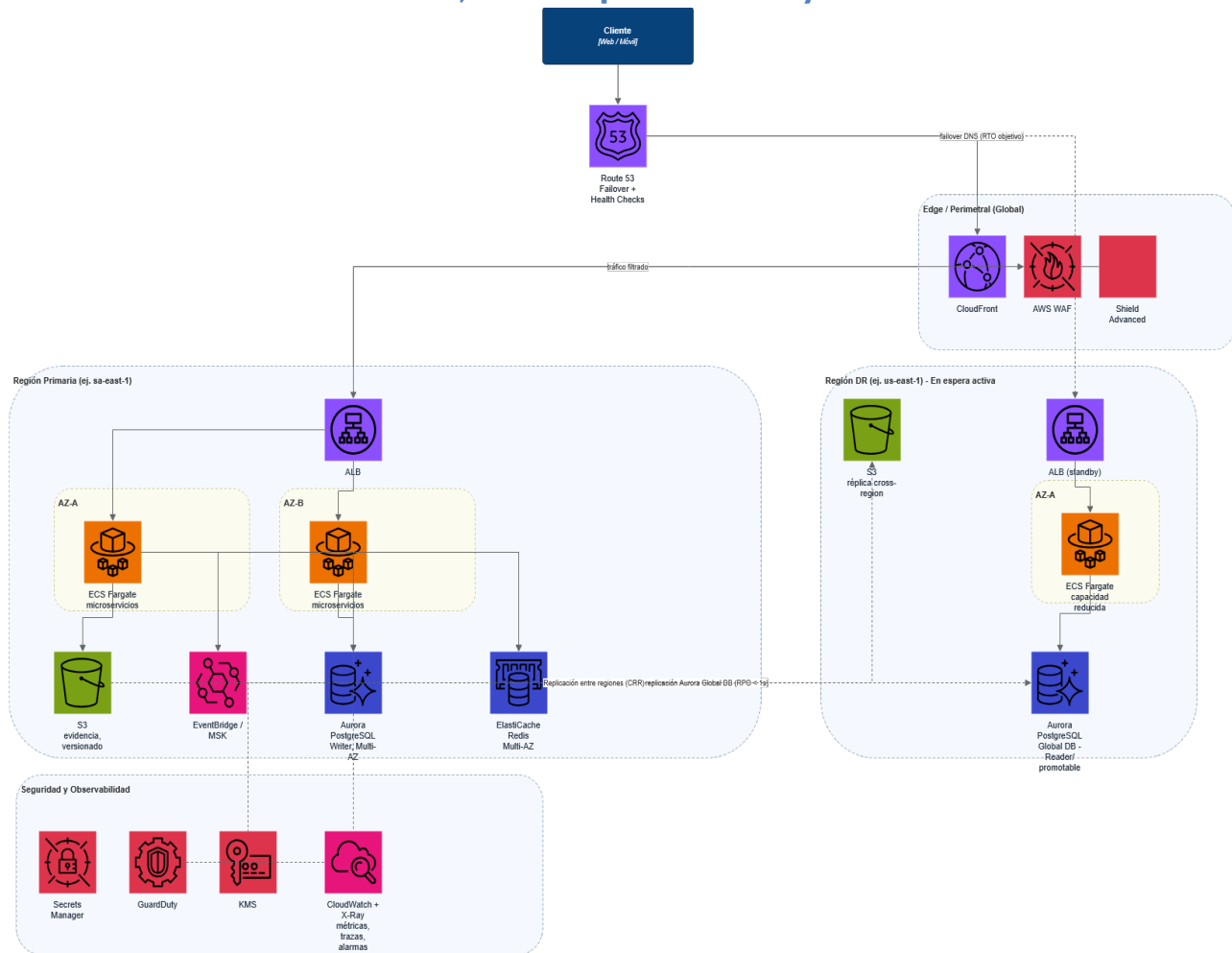
correspondiente en una tabla *outbox* local (patrón **Transactional Outbox**, que evita el problema de doble escritura entre base de datos y bus de eventos).

2. Un proceso (*Debezium-style change data capture* o un *publisher* propio) lee la tabla *outbox* y publica el evento en el **bus de eventos** (EventBridge/MSK).
 3. El **Servicio de Auditoría** consume estos eventos de forma asíncrona y los persiste en una tabla **append-only** en Aurora PostgreSQL, donde cada registro incluye el *hash* criptográfico del registro anterior (**hash chaining**), permitiendo detectar cualquier alteración retroactiva del log – una propiedad equivalente a la de un *ledger*.
 4. Para **clientes frecuentes**, se agrega una vista materializada / tabla de resumen (patrón **CQRS**, con el modelo de lectura separado del de escritura) que pre-agrega las últimas N acciones por cliente, evitando escanear el log completo en consultas de soporte o cumplimiento.
- *Justificación 1:* El patrón Outbox garantiza consistencia entre el estado transaccional y el evento de auditoría publicado, sin necesitar transacciones distribuidas (2PC) entre la base de datos y el bus de eventos.
 - *Justificación 2:* El *hash chaining* aporta evidencia de integridad (verificable ante un auditor o el regulador) sin depender de un servicio propietario como Amazon QLDB, que AWS discontinuó (fin de soporte el 31 de julio de 2025).

Arquitectura de acceso a datos

- **Patrón Adapter / Anti-Corruption Layer:** cada integración externa (Core Bancario, sistema complementario, red interbancaria) tiene un adaptador propio que traduce su modelo externo al modelo de dominio interno. Si el Core cambia su contrato, solo se modifica el adaptador correspondiente.
- **CQRS** entre el modelo transaccional (escritura, Aurora PostgreSQL) y el modelo de auditoría/reportería (lectura, vistas materializadas), evitando que consultas analíticas compitan por recursos con las transferencias.
- **Cache-aside** con Redis para consultas de solo lectura de alta frecuencia (saldo, últimos movimientos), con invalidación dirigida por evento (cuando se registra una nueva transferencia, se invalida la entrada de cache del cliente afectado).
- Persistencia poliglota: relacional (Aurora) para datos transaccionales con integridad referencial fuerte; almacenamiento de objetos (S3) para evidencia no estructurada (documentos, *selfies* de onboarding), cifrada con KMS y con políticas de *retention/lifecycle*.

Infraestructura cloud AWS, alta disponibilidad y DR



Infraestructura AWS – Alta Disponibilidad y Disaster Recovery (Para mayor detalle o ver mejor la imagen dirigirse a la imagen que está en el mismo repositorio o al draw.io)

Alta disponibilidad y tolerancia a fallos

- Todos los servicios de cómputo (**ECS Fargate**) se despliegan en **al menos dos zonas de disponibilidad (Multi-AZ)** dentro de la región primaria, detrás de un *Application Load Balancer* con *health checks* activos y auto-reemplazo de tareas no saludables.
- **Aurora PostgreSQL Multi-AZ** con réplica sincrónica para failover automático (segundos) ante falla de la instancia primaria.
- **ElastiCache Redis Multi-AZ** con failover automático.
- Patrones de resiliencia a nivel de aplicación (Circuit Breaker, retry con *backoff*, timeout, *bulkhead* para aislar pools de conexión por dependencia) descritos en la sección de Componentes.
- *Auto Scaling* basado en métricas de CPU/latencia/profundidad de cola para los servicios de mayor variabilidad de tráfico (Movimientos, Notificaciones).

Recuperación ante desastres (DR)

- Estrategia **warm standby multi-región**: una región DR mantiene infraestructura desplegada a capacidad reducida, lista para escalar rápidamente ante un failover regional.
- **Aurora Global Database** replica la base transaccional hacia la región DR con **RPO típico menor a 1 segundo**; en caso de desastre regional, la réplica se promueve a *writer* (RTO del orden de minutos).
- **S3 Cross-Region Replication** para evidencia de onboarding y documentos.
- **Route 53** con *health checks* y *routing* de *failover* dirige el tráfico automáticamente a la región DR si la primaria deja de responder.
- *Justificación de warm standby (vs. activo-activo)*: activo-activo minimiza aún más el RTO, pero duplica el costo operativo de forma permanente y añade complejidad de resolución de conflictos de escritura; para un sistema de banca digital con RTO objetivo en minutos (no segundos), *warm standby* es el punto óptimo costo/beneficio, y es una decisión revisable si el negocio define un RTO más agresivo.
- *Justificación 2*: Aurora Global Database desacopla la estrategia de DR de la lógica de aplicación (la replicación ocurre a nivel de motor de base de datos), evitando tener que construir mecanismos de sincronización propios entre regiones.

Seguridad

- **Perímetro**: CloudFront + AWS WAF (reglas OWASP Top 10 gestionadas) + AWS Shield Advanced contra DDoS.
- **En tránsito**: TLS 1.2+ en todos los saltos, incluyendo mTLS entre el API Gateway/BFF y los microservicios internos.
- **En reposo**: cifrado con AWS KMS (llaves administradas, rotación automática) para Aurora, S3 y ElastiCache; los datos biométricos y documentos de identidad se cifran con una *customer-managed key* dedicada, separada de la llave de datos transaccionales, para poder rotar/revocar acceso de forma independiente.
- **Gestión de secretos**: AWS Secrets Manager para credenciales de base de datos y llaves de integración con terceros, con rotación automática.
- **Identidad y acceso a infraestructura**: IAM con principio de mínimo privilegio, roles por servicio (no credenciales estáticas), y *Service Control Policies* a nivel de organización.
- **Detección**: Amazon GuardDuty para detección de amenazas (comportamiento anómalo, IPs maliciosas) y AWS Security Hub para consolidar hallazgos de cumplimiento.
- **Aplicativo**: validación de entrada en cada capa (Gateway, BFF, microservicio), *rate limiting* por cliente/IP, y *step-up authentication* biométrica para operaciones de alto riesgo (transferencias sobre cierto monto, cambios de dispositivo).

Monitoreo, excelencia operativa y auto-healing

- **Observabilidad:** Amazon CloudWatch (métricas y logs centralizados) + AWS X-Ray (trazas distribuidas *end-to-end*, esenciales para diagnosticar latencia en la cadena BFF → Orchestration → Adapter → Core Bancario).
- **Alarmas** sobre SLIs clave (latencia p95/p99, tasa de error 5xx, profundidad de *circuit breaker* abierto, *lag* de consumo del bus de eventos) con notificación a *on-call* vía Amazon SNS/Chatbot.
- **Auto-healing:** *health checks* de ALB reemplazan automáticamente tareas de ECS no saludables; Aurora y ElastiCache hacen failover automático Multi-AZ; *Auto Scaling* corrige tanto sub- como sobre-provisionamiento.
- **Excelencia operativa:** *Infrastructure as Code* (Terraform/CloudFormation) para despliegues reproducibles, *pipelines* de CI/CD con *canary/blue-green deployment* para reducir riesgo de despliegue, y *runbooks* documentados para los escenarios de incidente más probables (caída del Core Bancario, degradación del IdP, failover regional).
- **Auditoría de infraestructura:** AWS CloudTrail para trazabilidad de cambios sobre los propios recursos de AWS (quién cambió qué), complementario a la auditoría de negocio descrita anteriormente.

Gestión de costos

- **Cómputo:** ECS Fargate para servicios administrados por nosotros con tráfico variable, complementado con *Compute Savings Plans* para la capacidad base (*steady-state*)
- **Base de datos:** evaluar **Aurora Serverless v2** para entornos de menor tráfico (o no productivos) para pagar por capacidad realmente consumida en lugar de instancias fijas.
- **DR como warm standby** (no activo-activo) es en sí mismo una decisión de costos: mantiene el RTO en minutos sin duplicar el 100% del gasto de cómputo de forma permanente.
- **Almacenamiento:** políticas de *lifecycle* en S3 (paso a Glacier/Deep Archive para evidencia de onboarding con antigüedad definida por la política de retención regulatoria).
- **Red:** *caching* en API Gateway y CloudFront para reducir invocaciones redundantes al backend; NAT Gateway compartido por VPC/AZ en lugar de uno por servicio.
- **Gobierno de costos:** estrategia de *tagging* obligatorio por dominio/servicio para *cost allocation*, y AWS Cost Anomaly Detection para alertar desviaciones inesperadas.

Arquitectura desacoplada, reusable y extensible

La arquitectura fue diseñada explícitamente para permitir estos escenarios de evolución sin retrabajo estructural:

- **Nuevo canal (p. ej. asistente conversacional o *smartwatch*):** se agrega un nuevo BFF que reutiliza los mismos microservicios de dominio, sin duplicar lógica de negocio.

- **Nuevo consumidor de eventos (p. ej. motor de scoring antifraude):** se suscribe al bus de eventos existente sin modificar a los productores (Transferencias, Onboarding).
- **Cambio de proveedor biométrico o de IdP:** al estar aislados detrás de un adaptador (Servicio de Identidad / Servicio de Onboarding), el cambio de proveedor no impacta a los BFF ni a los microservicios de negocio.
- **Contratos versionados:** las APIs expuestas por el Gateway siguen especificación OpenAPI versionada (/v1, /v2), y los eventos del bus siguen contratos AsyncAPI, permitiendo evolución sin *breaking changes* para consumidores existentes.

Conclusiones

La arquitectura propuesta cumple los requerimientos funcionales del reto (consulta de movimientos, transferencias propias e interbancarias, onboarding biométrico, notificaciones multicanal) y los no funcionales exigidos explícitamente: alta disponibilidad y tolerancia a fallos (Multi-AZ + patrones de resiliencia), recuperación ante desastres (warm standby multi-región con Aurora Global Database), seguridad (defensa en profundidad, OAuth 2.0 + PKCE + WebAuthn, cifrado en tránsito/reposo), monitoreo y excelencia operativa (CloudWatch/X-Ray, auto-healing, IaC), y manejo de costos.

El diseño prioriza el desacoplamiento (microservicios de responsabilidad única, bus de eventos, patrón Adapter/ACL hacia sistemas legados) y la reutilización (BFF por canal, contratos versionados), de forma que BP pueda incorporar nuevos canales, proveedores o capacidades (por ejemplo, un motor antifraude) con impacto mínimo sobre los componentes existentes.

Gracias, Andrés Eduardo Cárdenas Jaramillo -- Arquitecto de Software.